Data Representation & APIs

Bit by Bit Coding - Term 2 | Week 12

Key Concepts

- Binary: computers store everything as bits (0 and 1); each place is a power of 2.
- Hexadecimal: base-16 using 0-9 and A-F; a compact way to show binary.
- ASCII/Unicode: codes that map numbers to characters (Unicode covers all languages).
- An API lets programs talk to a service over the web.
- The requests library fetches data; JSON is the common data format.
- REST: a style where URLs represent resources and use HTTP verbs (GET, POST).

Syntax Reference

Binary & Hexadecimal

```
print(bin(10))          # 0b1010  (8+2)
print(hex(255))         # 0xff
print(int("1010", 2))   # 10  (binary -> int)
print(int("ff", 16))    # 255 (hex -> int)
```

ASCII / Unicode

```
print(ord("A"))    # 65
print(chr(65))     # 'A'
print(ord("a"))    # 97
```

API with requests

```
import requests

url = "https://api.example.com/data"
response = requests.get(url)
print(response.status_code)   # 200 = OK

data = response.json()       # parse JSON -> dict/list
print(data["title"])
```

Working with JSON

```
import json

# Python dict -> JSON string
text = json.dumps({"name": "Ali", "age": 15})
# JSON string -> Python object
obj = json.loads('{"name": "Ali"}')
print(obj["name"])           # Ali
```

Common Patterns

- Always check status_code == 200 before using the data.
- Parse with .json(), then access the result like normal Python data.
- Add parameters: requests.get(url, params={"q": "python"}).

Tips & Common Mistakes

- requests needs an internet connection and the package installed.
- Forgetting .json() leaves the response as a string you can't index.
- Binary is base-2 (0-1); hex is base-16 (0-F). Don't confuse them.
- 200 OK = success; 404 = not found; 500 = server error.